



Version 9.0-FR, Avril 2025: (basé sur la version AsciiDoc)

Table des matières

1. Introduction et Objectifs	2
1.1. Aperçu des spécifications	2
1.2. Objectifs de Qualité	2
1.3. Parties prenantes	3
2. Contraintes d'Architecture	5
3. Contexte et périmètre	6
3.1. Contexte métier	6
3.2. Contexte Technique	7
4. Stratégie de solution	8
5. Vue en Briques	9
5.1. Niveau 1 : Système global Boîte blanche	11
5.1.1. <Nom boîte noire 1>	12
5.1.2. <Nom boîte noire 2>	12
5.1.3. <Nom boîte noire n>	12
5.1.4. <Nom interface 1>	12
5.1.5. <Nom interface m>	12
5.2. Niveau 2	12
5.2.1. Boîte blanche <brique 1>	13
5.2.2. Boîte blanche <brique 2>	13
5.2.3. Boîte blanche <brique n>	13
6. Vue Exécution	14
6.1. <Scénario d'exécution 1>	14
6.2. <Scénario d'exécution 2>	15
6.3.	15
6.4. <Scénario d'exécution n>	15
7. Vue Déploiement	16
7.1. Infrastructure Niveau 1	17
7.2. Infrastructure Niveau 2	17
7.2.1. <Infrastructure Element 1>	17
7.2.2. <Infrastructure Element 2>	17
7.2.3. <Infrastructure Element n>	17
8. Concepts transversaux	18
8.1. <Concept 1>	19
8.2. <Concept 2>	19
8.3. <Concept n>	19
9. Décisions d'architecture	20
10. Critères de qualité	21
10.1. Exigences de qualité - Vue d'ensemble	21

10.2. Scénarios Qualité	22
11. Risques et Dettes techniques	24
12. Glossaire	25

A propos d'arc42

arc42, le modèle de documentation de l'architecture des logiciels et des systèmes.

Version du modèle 9.0-FR. (basé sur la version AsciiDoc), Avril 2025

Créé, maintenu et © par Dr. Peter Hruschka, Dr. Gernot Starke et les contributeurs. Voir <https://arc42.org>.

NOTE

Cette version du modèle contient de l'aide et des explications. Elle est utilisée pour se familiariser avec arc42 et comprendre les concepts. Pour la documentation de votre propre système, il est préférable d'utiliser la version *simple*.

Chapter 1. Introduction et Objectifs

Décrit les spécifications importantes et les facteurs déterminants que les architectes logiciels et l'équipe de développement doivent prendre en compte. Il s'agit notamment

- des objectifs métier sous-jacents,
- des caractéristiques essentielles,
- des spécifications fonctionnelles essentielles,
- des objectifs de qualité pour l'architecture et
- des parties prenantes concernées et leurs attentes

1.1. Aperçu des spécifications

Contenu

Brève description des spécifications fonctionnelles, des facteurs déterminants, de l'extrait (ou du résumé) des spécifications. Liens vers (en espérant qu'ils existent) les documents décrivant les spécifications (avec le numéro de version et des informations sur l'endroit où les trouver).

Motivation

Du point de vue des utilisateurs, un système est créé ou modifié pour améliorer le soutien d'une activité métier et/ou améliorer la qualité.

Représentation

Brève description textuelle, probablement sous forme de tableau de cas d'utilisation. S'il existe des documents décrivant les spécifications, cet aperçu général doit s'y référer.

Ces extraits doivent être aussi courts que possible. Trouver un équilibre entre la lisibilité de ce document et la redondance potentielle par rapport aux documents décrivant les spécifications.

Informations supplémentaires

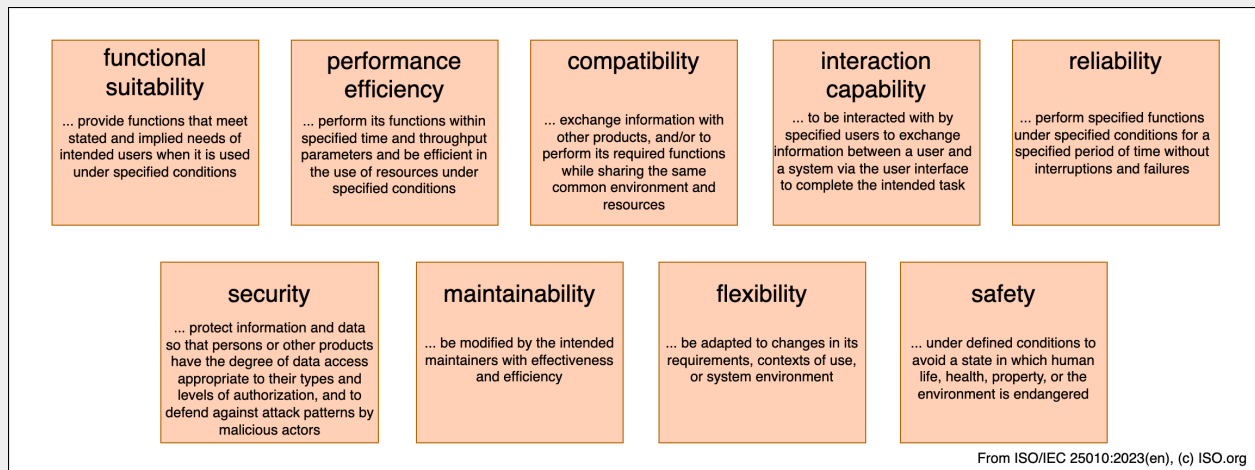
Voir [Introduction and Goals](#) dans la documentation arc42.

1.2. Objectifs de Qualité

Contenu

Les trois (maximum cinq) principaux objectifs de qualité pour l'architecture dont la réalisation est de la plus haute importance pour les principaux acteurs. Nous parlons bien d'objectifs de qualité pour l'architecture. Ne les confondez pas avec les objectifs du projet. Ils ne sont pas nécessairement identiques.

Voici un aperçu des sujets potentiels (basé sur la norme ISO 25010) :



Motivation

Vous devez connaître les objectifs de qualité de vos principales parties prenantes, car ils influenceront les décisions architecturales fondamentales. Veillez à être très concret sur ces qualités, évitez les mots à la mode. En tant qu'architecte, vous ne savez pas comment la qualité de votre travail sera jugée...

Représentation

Un tableau avec des objectifs de qualité et des scénarios concrets, classés par ordre de priorité

1.3. Parties prenantes

Contenu

Aperçu explicite des parties prenantes du système, c'est-à-dire toute personne, rôle ou organisation qui

- doit connaître l'architecture
- doit être convaincue de l'architecture
- doit travailler avec l'architecture ou avec le code
- a besoin de la documentation d'architecture pour son travail
- doit prendre des décisions concernant le système ou son développement

Motivation

Vous devez connaître toutes les parties impliquées dans le développement du système ou concernées par celui-ci. Sinon, vous risquez d'avoir de mauvaises surprises plus tard dans le processus de développement. Ces parties prenantes déterminent l'étendue et le niveau de détail de votre travail et de ses résultats.

Représentation

Tableau avec les noms des rôles, les noms des personnes et leurs attentes par rapport à

l'architecture et à sa documentation.

Rôle/Nom	Contact	Attentes
<Role-1>	<Contact-1>	<Attente-1>
<Role-2>	<Contact-2>	<Attente-2>

Chapter 2. Contraintes d'Architecture

Contenu

Toute spécification qui limite la liberté des architectes logiciels dans leurs décisions de conception et de mise en œuvre ou dans leurs décisions concernant le processus de développement. Ces contraintes vont parfois au-delà des systèmes individuels et sont valables pour l'ensemble des organisations et des entreprises.

Motivation

Les architectes doivent savoir exactement où ils sont libres dans leurs décisions de conception et où ils doivent respecter des contraintes. Les contraintes doivent toujours être prises en compte ; elles peuvent toutefois être négociables.

Représentation

Tableaux simples de contraintes avec explications. Si nécessaire, vous pouvez les subdiviser en contraintes techniques, contraintes organisationnelles, politiques et conventions (par exemple, règles de développement ou de gestion de versions, conventions de documentation ou de nomenclature).

Informations supplémentaires

Voir [Architecture Constraints](#) dans la documentation arc42.

Chapter 3. Contexte et périmètre

Contenu

Le contexte et le périmètre - comme son nom l'indique - délimitent votre système (c'est-à-dire votre périmètre) de tous ses partenaires de communication (systèmes voisins et utilisateurs, c'est-à-dire le contexte de votre système). Il spécifie ainsi les interfaces externes.

Si nécessaire, différencier le contexte métier (entrées et sorties spécifiques au domaine) du contexte technique (canaux, protocoles, matériel).

Motivation

Les interfaces de domaine et les interfaces techniques avec les partenaires de communication font partie des aspects les plus critiques de votre système. Assurez-vous de bien les comprendre.

Représentation

Diverses options :

- Diagrammes de contexte
- Listes des partenaires de communication et de leurs interfaces.

Informations supplémentaires

Voir [Context and Scope](#) dans la documentation arc42.

3.1. Contexte métier

Contenu

Spécification de **tous** les partenaires de communication (utilisateurs, systèmes informatiques, ...) avec des explications sur les entrées et les sorties ou les interfaces spécifiques au domaine. En option, vous pouvez ajouter des formats ou des protocoles de communication spécifiques au domaine.

Motivation

Toutes les parties prenantes doivent comprendre quelles données sont échangées avec l'environnement du système.

Représentation

Toutes sortes de diagrammes qui montrent le système comme une boîte noire et spécifient les interfaces du domaine avec les partenaires de communication.

Vous pouvez également (ou en plus) utiliser un tableau. Le titre du tableau est le nom de votre système, les trois colonnes contiennent le nom du partenaire de communication, les entrées et les sorties.

<Schéma ou tableau>

<éventuellement : Explication des interfaces de domaines externes>

3.2. Contexte Technique

Contenu

Interfaces techniques (canaux et supports de transmission) reliant votre système à son environnement. En outre, il faut établir une correspondance entre les entrées/sorties spécifiques au domaine et les canaux, c'est-à-dire expliquer quelles Entrées/Sorties utilisent quel canal.

Motivation

De nombreuses parties prenantes prennent des décisions architecturales basées sur les interfaces techniques entre le système et son contexte. Ce sont surtout les concepteurs d'infrastructure ou d'équipements informatiques qui décident de ces interfaces techniques.

Représentation

Par exemple, un diagramme de déploiement UML décrivant les canaux vers les systèmes voisins, ainsi qu'un tableau de correspondance montrant les relations entre les canaux et les entrées/sorties.

<Schéma ou tableau>

<en option : Explication des interfaces techniques>

<Correspondance des entrées/sorties aux canaux>

Chapter 4. Stratégie de solution

Contenu

Un bref résumé et une explication des décisions fondamentales et des stratégies de solution qui façonnent l'architecture du système. Il comprend

- les décisions technologiques
- les décisions relatives à la décomposition du système au niveau le plus élevé, par exemple l'utilisation d'un modèle architectural ou d'un modèle de conception
- les décisions sur la manière d'atteindre les principaux objectifs de qualité
- les décisions organisationnelles pertinentes, par exemple la sélection d'un processus de développement ou la délégation de certaines tâches à des tierces personnes.

Motivation

Ces décisions constituent les pierres angulaires de votre architecture. Elles constituent le fondement de nombreuses autres décisions détaillées ou règles de mise en œuvre.

Représentation

Les explications relatives à ces décisions clés doivent être brèves.

Motiver ce qui a été décidé et pourquoi cela a été décidé de cette manière, sur la base de l'énoncé du problème, des objectifs de qualité et des principales contraintes.

Pour documenter la manière d'atteindre les principaux objectifs de qualité, vous pouvez utiliser le tableau suivant :

Objectif de qualité	Scénario	Approche de la solution	Lien vers les détails
<Objectif-Q1>	<Scénario-1>	<Solution-1>	<Lien-1>
<Objectif-Q2>	<Scénario-2>	<Solution-2>	<Lien-2>

Informations supplémentaires

Voir [Solution Strategy](#) dans la documentation arc42.

Chapter 5. Vue en Briques

Contenu

La vue en briques montre la décomposition statique du système en briques (modules, composants, sous-systèmes, classes, interfaces, paquets, bibliothèques, cadres, couches, partitions, niveaux, fonctions, macros, opérations, structures de données, ...) ainsi que leurs dépendances (relations, associations, ...).

Cette vue est obligatoire pour toute documentation sur l'architecture. Par analogie avec une maison, il s'agit du *plan de masse*.

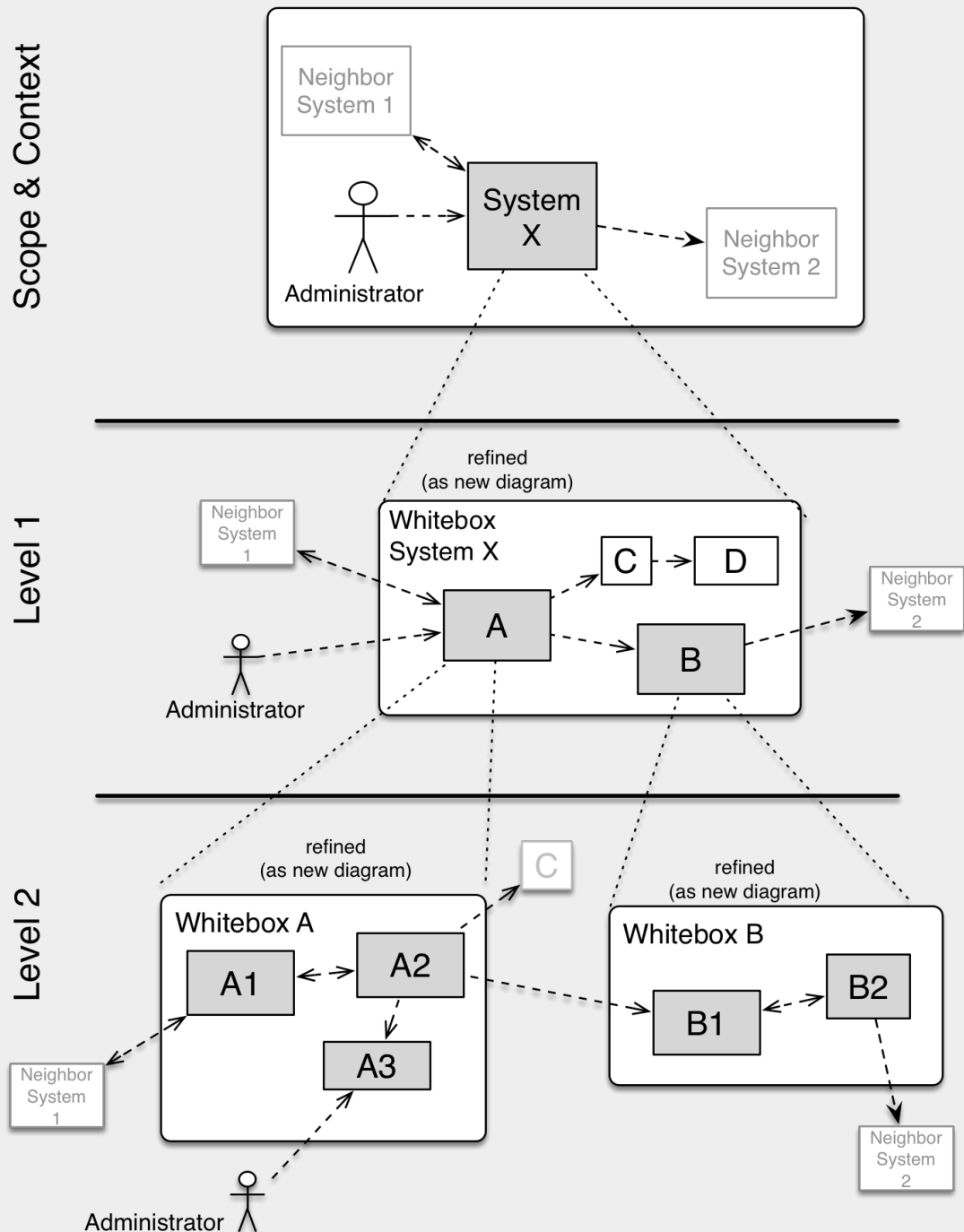
Motivation

Maintenir une vue d'ensemble de votre code en rendant sa structure compréhensible grâce à l'abstraction.

Cela vous permet de communiquer avec toute partie prenante à un niveau abstrait sans divulguer les détails de l'implémentation.

Représentation

La vue en briques est une collection hiérarchique de boîtes noires et de boîtes blanches (voir figure ci-dessous) et de leurs descriptions.



Niveau Contexte et Périmètre (Niveau 0) a été décrit dans la section [Context and Scope](#). C'est pourquoi il n'est pas décrit ici, mais vous pouvez éventuellement insérer un lien vers [Context and Scope](#).

Niveau 1 est la description en boîte blanche du système global ainsi que la description en boîte noire de toutes les briques qu'il contient.

Niveau 2 est un zoom sur certaines briques du niveau 1. Il contient donc la description en boîte blanche de certaines briques du niveau 1, ainsi que la description en boîte noire de leurs briques internes.

Informations supplémentaires

Voir [Building Block View](#) dans la documentation arc42.

5.1. Niveau 1 : Système global Boîte blanche

Vous décrivez ici la décomposition du système global à l'aide du modèle de boîte blanche suivant. Il contient

- un schéma d'ensemble
- une motivation pour la décomposition
- des descriptions boîte noire des briques contenues. Pour cela, nous vous proposons des alternatives :
 - utiliser *un* tableau pour une vue d'ensemble courte et pragmatique de tous les éléments contenus et de leurs interfaces
 - utiliser une liste de descriptions boîte noire des briques conformément au modèle de boîte noire (voir ci-dessous). Selon l'outil choisi, cette liste peut prendre la forme de sous-chapitres (dans les fichiers texte), de sous-pages (dans un wiki) ou d'éléments imbriqués (dans un outil de modélisation).
- (facultatif :) interfaces importantes, qui ne sont pas expliquées dans les modèles boîte noire d'une brique, mais qui sont très importantes pour la compréhension de la boîte blanche. Puisqu'il existe de nombreuses façons de spécifier les interfaces, pourquoi ne pas fournir un modèle spécifique pour celles-ci ? Dans le pire des cas, vous devez spécifier et décrire la syntaxe, la sémantique, les protocoles, la gestion des erreurs, les restrictions, les versions, les qualités, les compatibilités nécessaires et bien d'autres choses encore. Dans le meilleur des cas, vous pourrez vous contenter d'exemples ou de simples signatures.

<Schéma d'ensemble>

Motivation

<texte explicatif>

Briques contenues

<Description de la brique contenue (boîte noire)>

Interfaces Importantes

<Description des interfaces importantes>

Insérez vos explications sur les boîtes noires du niveau 1 :

Si vous utilisez la forme tabulaire, vous ne décrirez vos boîtes noires qu'avec leur nom et leur responsabilité selon le schéma suivant :

Nom	Responsabilité
<i><boîte noire 1></i>	<i><Texte></i>
<i><boîte noire 2></i>	<i><Texte></i>

Si vous utilisez une liste de descriptions de boîtes noires, vous devez remplir un modèle de

boîte noire distinct pour chaque brique importante. Son titre est le nom de la boîte noire.

5.1.1. <Nom boîte noire 1>

Vous décrivez ici la <boîte noire 1> selon le modèle de boîte noire suivant :

- Objectif/Responsabilité
- Interface(s), lorsqu'elle(s) n'est (ne sont) pas extraite(s) en tant que paragraphe(s) séparé(s). Ces interfaces peuvent inclure des qualités et des caractéristiques de performance.
- (Facultatif) Caractéristiques de qualité/performance de la boîte noire, par exemple disponibilité, comportement en cours d'exécution,
- (Facultatif) Emplacement du répertoire/fichier
- (Facultatif) Exigences respectées (si vous avez besoin d'une traçabilité vers les exigences)
- (Facultatif) Questions ouvertes/problèmes/risques

<Objectif/Responsabilité>

<Interface(s)>

<(Facultatif) Caractéristiques de qualité/performance>

<(Facultatif) Emplacement du répertoire/fichier>

<(Facultatif) Exigences respectées>

<(Facultatif) Questions ouvertes/problèmes/risques>

5.1.2. <Nom boîte noire 2>

<template boîte noire>

5.1.3. <Nom boîte noire n>

<template boîte noire>

5.1.4. <Nom interface 1>

...

5.1.5. <Nom interface m>

5.2. Niveau 2

Ici, vous pouvez spécifier la structure interne de (certaines) briques du niveau 1 sous forme de boîtes blanches.

Vous devez décider quels éléments de votre système sont suffisamment importants pour justifier une description aussi détaillée. Préférez la pertinence à l'exhaustivité. Spécifiez les éléments importants, surprenants, risqués, complexes ou volatils. Laissez de côté les éléments normaux, simples, ennuyeux ou standardisés de votre système.

Si vous avez besoin de niveaux plus détaillés de votre architecture, c'est-à-dire des niveaux 3, 4 et ainsi de suite, veuillez copier cette partie d'arc42 pour les niveaux supplémentaires.

5.2.1. Boîte blanche <*brique 1*>

...décrit la structure interne de la *brique 1*.

<template boîte blanche>

5.2.2. Boîte blanche <*brique 2*>

<template boîte blanche>

...

5.2.3. Boîte blanche <*brique n*>

<template boîte blanche>

Chapter 6. Vue Exécution

Contenu

La vue d'exécution décrit le comportement concret et les interactions des briques du système sous la forme de scénarios dans les domaines suivants :

- cas d'utilisation ou caractéristiques importants : comment les briques les exécutent-ils ?
- interactions aux interfaces externes critiques : comment les briques coopèrent-elles avec les utilisateurs et les systèmes voisins ?
- fonctionnement et administration : lancement, démarrage, arrêt
- scénarios d'erreur et d'exception

Remarque : Le critère principal pour le choix des scénarios possibles (séquences, flux de travail) est leur **importance architecturale**. Il n'est **pas** important de décrire un grand nombre de scénarios. Vous devez plutôt documenter une sélection représentative.

Motivation

Vous devez comprendre comment les (instances des) briques de votre système effectuent leur travail et communiquent au moment de l'exécution. Vous capturerez principalement des scénarios dans votre documentation afin de communiquer votre architecture aux parties prenantes qui sont moins disposées ou capables de lire et de comprendre les modèles statiques (vue en briques, vue déploiement).

Représentation

Il existe de nombreuses notations pour décrire les scénarios, par exemple

- liste numérotée d'étapes (en langage naturel)
- diagrammes d'activités ou de flux
- diagrammes de séquence
- BPMN ou EPC (chaînes de processus d'événements)
- machines d'états
- ...

Informations supplémentaires

Voir [Runtime View](#) dans la documentation arc42.

6.1. <Scénario d'exécution 1>

- *<insérer un diagramme d'exécution ou une description textuelle du scénario>*
- *<insérer une description des aspects notables des interactions entre les instances des briques représentées dans ce diagramme.>*

6.2. <Scénario d'exécution 2>

6.3. ...

6.4. <Scénario d'exécution n>

Chapter 7. Vue Déploiement

Contenu

La vue déploiement décrit :

1. l'infrastructure technique utilisée pour exécuter votre système, avec des éléments d'infrastructure tels que les emplacements géographiques, les environnements, les serveurs, les processeurs, les canaux et les topologies de réseau, ainsi que d'autres éléments d'infrastructure, et
2. la correspondance entre les briques (logicielles) et les éléments d'infrastructure.

Les systèmes sont souvent exécutés dans différents environnements, par exemple l'environnement de développement, l'environnement de test, l'environnement de production. Dans ce cas, vous devez documenter tous les environnements importants.

Documenter en particulier une vue déploiement si votre logiciel est exécuté en tant que système distribué avec plus d'un ordinateur, processeur, serveur ou conteneur ou lorsque vous concevez et construisez vos propres processeurs et puces matérielles.

D'un point de vue logiciel, il suffit de décrire uniquement les éléments d'une infrastructure qui sont nécessaires pour montrer le déploiement de vos briques. Les architectes matériel peuvent aller plus loin et décrire une infrastructure à n'importe quel niveau de détail.

Motivation

Les logiciels ne fonctionnent pas sans matériel. Cette infrastructure sous-jacente peut influencer et influencera un système et/ou certains concepts transversaux. Il est donc nécessaire de connaître l'infrastructure.

Représentation

Un diagramme de déploiement de haut niveau est peut-être déjà contenu dans la section 3.2. en tant que contexte technique avec votre propre infrastructure comme UNE boîte noire. Dans cette section, il est possible de zoomer sur cette boîte noire à l'aide de diagrammes de déploiement supplémentaires :

- UML propose des diagrammes de déploiement pour exprimer ce point de vue. Utilisez-les, probablement avec des diagrammes imbriqués, lorsque votre infrastructure est plus complexe.
- Lorsque vos parties prenantes (matérielles) préfèrent d'autres types de diagrammes plutôt qu'un diagramme de déploiement, laissez-les utiliser n'importe quel type de diagramme capable de montrer les nœuds et les canaux de l'infrastructure.

Plus d'informations

Voir [Vue Déploiement](#) dans la documentation arc42.

7.1. Infrastructure Niveau 1

Décrire (généralement à l'aide d'une combinaison de diagrammes, de tableaux et de textes) :

- distribution d'un système à plusieurs endroits, environnements, machines, processeurs, ... , ainsi que les connexions physiques entre eux
- justifications ou motivations importantes pour cette structure de déploiement
- les caractéristiques de qualité et/ou de performance de cette infrastructure
- la mise en correspondance des artefacts logiciels avec les éléments de cette infrastructure

Pour les environnements multiples ou les déploiements alternatifs, veuillez copier et adapter cette section d'arc42 pour tous les environnements concernés.

<Schéma d'ensemble>

Motivation

<explication sous forme de texte>

Caractéristiques de qualité et/ou de performance

<explication sous forme de texte>

Correspondance des briques vis à vis de l'infrastructure

<description de la correspondance>

7.2. Infrastructure Niveau 2

Ici, vous pouvez inclure la structure interne de (certains) éléments d'infrastructure du niveau 1.

Veuillez copier la structure du niveau 1 pour chaque élément sélectionné.

7.2.1. <Infrastructure Element 1>

<schéma + explication>

7.2.2. <Infrastructure Element 2>

<schéma + explication>

...

7.2.3. <Infrastructure Element n>

<schéma + explication>

Chapter 8. Concepts transversaux

Contenu

Cette section décrit les réglementations générales, principales et les idées de solutions qui sont importantes dans plusieurs parties (= transversales) de votre système. Ces concepts sont souvent liés à plusieurs briques. Ils peuvent inclure de nombreux sujets différents, tels que

- les modèles, particulièrement les modèles de domaine
- les modèles d'architecture ou de conception
- règles d'utilisation d'une technologie spécifique
- les décisions principales, souvent techniques, de nature globale (= transversale)
- règles de mise en œuvre

Motivation

Les concepts constituent la base de l'*intégrité conceptuelle* (cohérence, homogénéité) de l'architecture. Ils constituent donc une contribution importante à l'obtention des qualités internes de votre système.

Certains de ces concepts ne peuvent pas être attribués à des briques individuelles, par exemple la sécurité ou la fiabilité.

Représentation

La représentation peut être variée :

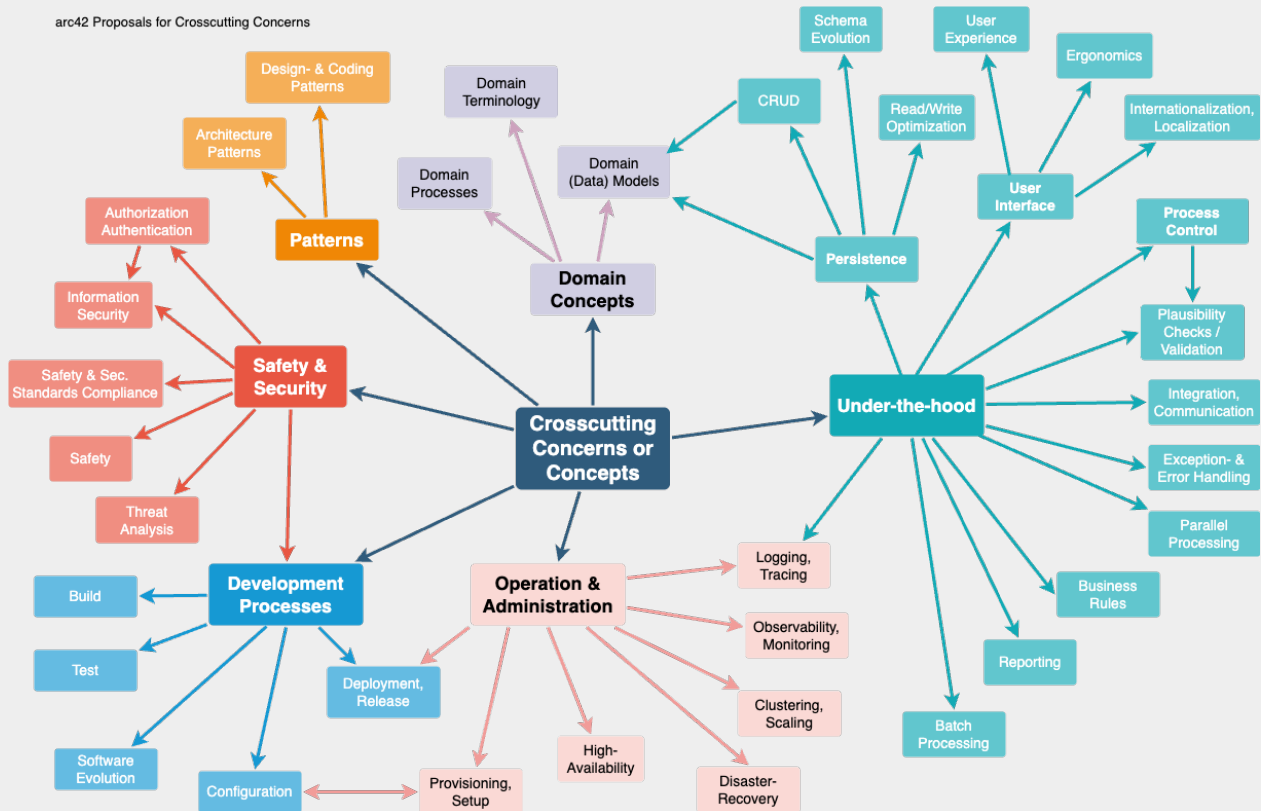
- des documents conceptuels avec n'importe quel type de structure
- des extraits de modèles transverses ou des scénarios utilisant les notations des vues de l'architecture
- des exemples de mise en œuvre, en particulier pour les concepts techniques
- référence à l'utilisation typique de frameworks standard (par exemple, l'utilisation d'Hibernate pour le mappage objet/relationnel)

Structure

Une structure potentielle (mais non obligatoire) pour cette section pourrait être la suivante :

- Concepts de domaine
- Concepts d'expérience utilisateur (UX)
- Concepts de fiabilité et de sécurité
- Architecture et modèles de conception
- "Sous le capot"
- Concepts de développement
- Concepts opérationnels

Remarque : il peut être difficile d'affecter des concepts individuels à un thème spécifique de cette liste.



Plus d'informations

Voir [Concepts](#) dans la documentation arc42.

8.1. <Concept 1>

<explication>

8.2. <Concept 2>

<explication>

...

8.3. <Concept n>

<explication>

Chapter 9. Décisions d'architecture

Contenu

Décisions architecturales importantes, coûteuses, à grande échelle ou risquées, y compris les justifications. Par "décisions", nous entendons la sélection d'une alternative sur la base de critères donnés.

Veillez faire preuve de discernement pour décider si une décision architecturale doit être documentée ici dans la section centrale ou s'il est préférable de la documenter localement (par exemple, dans le modèle de boîte blanche d'une brique).

Évitez la redondance. Reportez-vous à la section 4, où vous avez déjà pris les décisions les plus importantes de votre architecture.

Motivation

Les parties prenantes de votre système doivent pouvoir comprendre et retracer vos décisions.

Représentation

Diverses options :

- ADR ([Documenting Architecture Decisions](#)) pour chaque décision importante
- Liste ou tableau, classés par ordre d'importance et de conséquences ou :
- plus détaillé sous forme de sections séparées par décision

Informations supplémentaires

Voir [Architecture Decisions](#) dans la documentation arc42. Vous y trouverez des liens et des exemples sur les ADR.

Chapter 10. Critères de qualité

Contenu

Cette section contient toutes les exigences de qualité pertinentes.

Les plus importantes d'entre elles ont déjà été décrites au point 1.2 (objectifs de qualité) ; il convient donc de n'y faire référence qu'ici. Dans cette section, il convient également d'indiquer les exigences de qualité de moindre importance, qui n'entraîneront pas de risques élevés si elles ne sont pas entièrement satisfaites (quand même, il serait préférable qu'elles soient satisfaites).

Motivation

Comme les exigences de qualité ont une grande influence sur les décisions architecturales, vous devez savoir quelles sont les qualités réellement importantes pour vos parties prenantes, d'une manière spécifique et mesurable.

Informations supplémentaires

Voir [Quality Requirements](#) dans la documentation arc42. En plus, voir aussi [Q42 Quality Model](#) on <https://quality.arc42.org>.

10.1. Exigences de qualité - Vue d'ensemble

Contenu

Une vue d'ensemble ou un résumé des exigences de qualité.

Motivation

Nous rencontrons souvent des dizaines (voire des centaines) d'exigences de qualité détaillées. Dans cette section générale, vous devez essayer de les résumer, par exemple en décrivant des attributs de qualité (comme suggéré par <https://www.iso.org/obp/ui/#iso:std:iso-iec:25010:ed-2:v1:en> [ISO 25010:2023] ou <https://quality.arc42.org> [Q42]).

Si ces descriptions sommaires sont déjà suffisamment précises, spécifiques et mesurables, vous pouvez sauter la section 10.2.

Représentation

Utilisez un tableau simple dans lequel chaque ligne contient un attribut ou un sous-attribut et une brève description de l'exigence de qualité. Vous pouvez également utiliser une carte heuristique pour structurer ces exigences de qualité. Dans la littérature, l'idée d'un *arbre d'attributs de qualité* a également été décrite, qui place le terme générique « qualité » à la racine et utilise un raffinement arborescent du terme « qualité » avec les attributs, les sous-attributs et finalement les liens vers les scénarios (cf. section suivante). [Bass+21] a introduit le terme « Quality Attribute Utility Tree » pour cet *arbre d'attributs de qualité*.

10.2. Scénarios Qualité

Contenu

Les scénarios de qualité concrétisent les exigences de qualité et permettent de décider si elles sont satisfaites (au sens des critères d'acceptation). Veillez à ce que vos scénarios soient spécifiques et mesurables.

Deux types de scénarios sont particulièrement utiles

- Les scénarios d'utilisation (également appelés scénarios d'application ou scénarios de cas d'utilisation) décrivent la réaction du système en cours d'exécution à un certain stimulus. Cela inclut également les scénarios qui décrivent l'efficacité ou la performance du système. Exemple : Le système réagit à la demande d'un utilisateur en une seconde.
- Les scénarios de changement décrivent l'effet souhaité d'une modification ou d'une extension du système ou de son environnement immédiat. Exemple : Une fonctionnalité supplémentaire est mise en œuvre ou les exigences relatives à un attribut de qualité sont modifiées, et l'effort ou la durée du changement est mesuré.

Représentation

Les informations typiques pour les scénarios détaillés sont les suivantes :

Sous forme abrégée (privilégiée dans le modèle Q42) :

- **Contexte** : Quel type de système ou de composant, quel est l'environnement ou la situation ?
- **Source/Stimulus** : Qui ou quoi initie ou déclenche un comportement, une réaction ou une action.
- **Critères de mesure/d'acceptation** : Une réponse comprenant une *mesure* ou une *métrique*.

La forme longue des scénarios (privilégiée par le SEI et [Bass+21]) est plus détaillée et comprend les informations suivantes :

- **Identification du scénario** : Un identifiant unique pour le scénario.
- **Nom du scénario** : Un nom court et descriptif pour le scénario.
- **Source** : L'entité (utilisateur, système ou événement) qui initie le scénario.
- **Stimulus** : L'événement déclencheur ou la condition à laquelle le système doit répondre.
- **Environnement** : Le contexte opérationnel ou la condition dans laquelle le système subit le stimulus.
- **Artifact** : Les blocs de construction ou autres éléments du système affectés par le stimulus.
- **Réponse** : Le résultat ou le comportement du système en réaction au stimulus.
- **Mesure de la réponse** : Critères ou mesures permettant d'évaluer la réponse du système.

Exemples

Voir <https://quality.arc42.org> [le site web du modèle de qualité Q42] pour des exemples détaillés d'exigences de qualité.

Informations complémentaires

- Len Bass, Paul Clements, Rick Kazman : « Software Architecture in Practice », 4e édition, Addison-Wesley, 2022.

Chapter 11. Risques et Dettes techniques

Contenu

Une liste des risques ou des dettes techniques identifiés, classés par ordre de priorité

Motivation

“Risk management is project management for grown-ups” (Tim Lister, Atlantic Systems Guild.)

C'est la devise pour la détection et l'évaluation systématiques des risques et des dettes techniques dans l'architecture, dont auront besoin les acteurs de la gestion de projet (par exemple, les chefs de projet, les propriétaires de produits) dans le cadre de l'analyse globale des risques et de la planification de la mesure.

Représentation

Liste des risques et/ou des dettes techniques, comprenant probablement des mesures suggérées pour minimiser, atténuer ou éviter les risques ou réduire les dettes techniques.

Informations supplémentaires

Voir [Risks and Technical Debt](#) dans la documentation arc42.

Chapter 12. Glossaire

Contenu

Les termes techniques et métier les plus importants que vos parties prenantes utilisent lorsqu'elles discutent du système.

Le glossaire peut également servir de source pour les traductions si vous travaillez dans des équipes multilingues.

Motivation

Vous devez définir clairement vos termes, de manière à ce que toutes les parties prenantes

- aient une compréhension identique de ces termes
- n'utilisent pas de synonymes et d'homonymes

Représentation

Un tableau avec les colonnes <Terme> et <Définition>.

Potentiellement plus de colonnes au cas où vous auriez besoin de traductions.

Terme	Définition
<Terme-1>	<Définition-1>
<Terme-2>	<Définition-2>

Informations supplémentaires

Voir [Glossary](#) dans la documentation arc42.

Terme	Définition
<Terme-1>	<Définition-1>
<Terme-2>	<Définition-2>